



AUTOMATED REPAIR, JUDGED BY YOUR TESTS



Repairs the bug. Refuses the guess.

A guard for your test suite. When a regression ships, it restores the line, byte-exact, and commits — or refuses and tells you why. Zero tokens. Unattended.

Devieswar Kancheti · devkancheti4@gmail.com

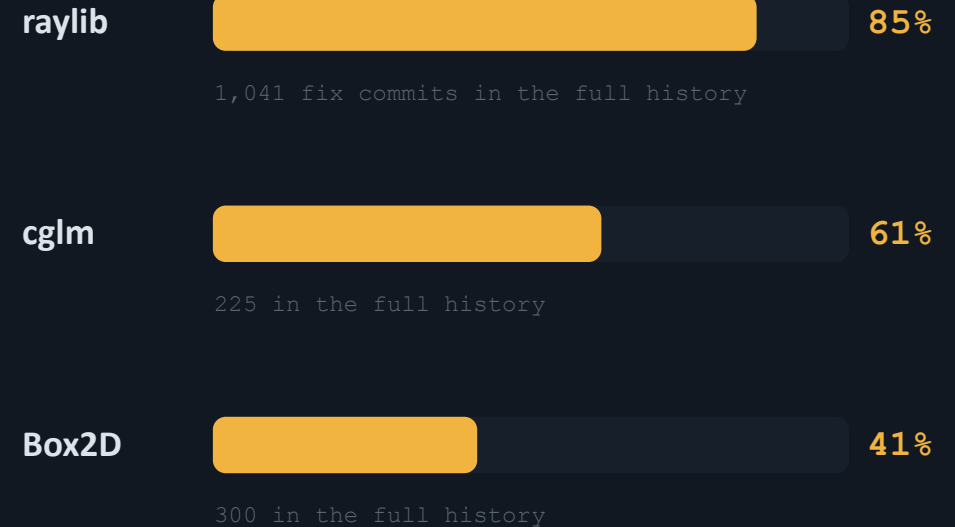
fluidfix · github.com/devkancheti4-design/fluidfix

Most regressions are one file and a few lines. Each one still costs an afternoon.

- A teammate tidies a formula. One operator flips. The suite goes red.
- Your tests already catch it. Nothing puts it back.
- So a person context-switches, or an agent reads files it does not need, to restore one line.
- Across a year that is the maintenance budget: not the hard bugs, the many small ones.

Share of real fix commits confined to one file

full git history, fix-worded commits, source files only



research/headtohead-2026-09-07/REACH.md

Two ways it gets fixed now. Both expensive.

A person

- Drops what they were doing to find one line.
- Reads the diff, reruns the suite, opens the PR, waits for review.
- Repeats it next week for the next one — the same shapes come back.

An agent

43,006 tokens and 73.8 s to restore one flipped sign

- Reads files it does not need to reach the line it does.
- A plausible fix that passes the tests can still be the wrong program — and it ships.
- Same defect, fluidfix: 0 tokens, 6.3 s, byte-exact. Both arms preserved.

measured head to head on the same real Box2D defect · [research/headtohead-2026-09-07](#)

It runs where your tests run. Red goes green, or it tells you why not.

Suite goes red

a regression is committed;
nobody has to notice



fluidfix finds the line

from your failing tests, in seconds



Your tests judge the fix

every candidate runs the whole
suite



Restored and committed

byte-exact, as fluidfix: restore
file:line

Or it refuses. A fix it cannot prove is never written: the tree stays byte-identical, exit code 2 for CI, and a report names what was tried and which test rejected it.

Deploys as a GitHub Action, a cron job, or a CI gate. · Python today; C/C++, Java and C# paths in alpha. · No model, no tokens, nothing leaves your machine.



WHY YOU CAN TRUST IT

A wrong repair cannot land.



Your tests are the only judge

No model decides. Every candidate runs your whole suite before a byte is kept.



Rejected means rolled back

A candidate that fails is restored byte for byte, line endings included.



Ambiguity is refused

Two fixes that both pass are two programs. It refuses, and writes the test that tells them apart, for you.



Crash-safe

Killed mid-write, the next run restores the original bytes first. Nothing to re-queue.

0

wrong repairs in 64 runs, Python and C, 2026-09-10

`research/maintenance-2026-09-10`

0 tokens

no model is invoked; nothing leaves your machine

`by construction`



Every incident you've had becomes a repair it makes for you, forever.

- Nine shapes of mechanical bug are repaired out of the box: flipped operators, off-by-one, swapped operands, min/max, booleans, comparisons.
- Each incident from your own history becomes a class of its own — one worked example, written down once.
- From then on every member of that class is restored automatically. The same shapes come back; the fix is already there.
- Your incident log stops being a cost centre and becomes the thing that pays.

1 → 4

one example from 2016 produced the correct repair for four real fixes, 2016–2023, four files

`cglm history · research/laws-2026-09-07/44`

9

shapes repaired with no teaching at all

`fluidfix kinds`

Every number has a file behind it.

26 / 27

byte-exact repairs, 33-bug benchmark

`docs/data/arm_a_full_results.json`

14 / 18

repaired across three real Python repositories
(78%)

`docs/data/*-after.log`

0

wrong repairs in 64 runs, Python and C

`research/maintenance-2026-09-10`

33 / 33

defects localised to the right file

`docs/data/arm_a_full_results.json`

46 runs

to a byte-exact repair inside Box2D, no class
written

`docs/BENCHMARK.md`

225 / 225

tests green on the shipping build

`full_suite_final2.log`

All measurements reproduce from the repository: github.com/devkancheti4-design/fluidfix

What it will not do, stated plainly.

It needs a test suite

pytest today; JUnit and a C test binary in alpha. A test that cannot see the defect cannot judge the repair.

One file at a time

Fixes that span files are out of scope today. On some engines that is most of them; on most services it is not.

Unknown shapes are refused

Until you show it one example, it leaves the bug alone and says so. That is the feature, not a gap.

It does not reason

It restores what it can prove and refuses the rest. Anything needing new information stays yours.

Start with the number you don't have yet.

OPEN SOURCE

Free

AGPL-3.0

Everything. All repair classes, every language path, unlimited repositories, no feature gate.

MEASUREMENT PILOT

\$5,000

six weeks

We run it on your history and your suite and produce the number: how many of your regressions are this shape, and what it would have saved. Credited in full against a licence.

COMMERCIAL LICENCE

\$12,000

per year

Closed-source use, up to 10 repositories, one language path, CI integration, the hotspots report.

STUDIO

\$40,000

per year

Unlimited repositories, all paths, and we author and maintain the repair classes from your own incident history.

No per-seat pricing: it runs on your machine and costs nothing per repair.

Twenty minutes. **Bring a repo.**

We point it at one repository you already have, live, and you watch it restore a real regression or refuse it and say why. Then we scope the pilot against your own history.

```
devkancheti4@gmail.com
```

```
github.com/devkancheti4-design/fluidfix · pypi.org/  
project/fluidfix
```

Repairs the bug. Refuses the guess.